

Playwright Interview Questions with Detailed Answers

1. What is Playwright?

Playwright is an open-source automation testing framework developed by Microsoft for end-to-end web application testing.

It supports:

- Chromium
- Firefox
- WebKit

Key Features

- Cross-browser testing
- Auto-wait mechanism
- Parallel execution
- Multi-tab and multi-window support
- Network interception
- API testing
- Mobile emulation
- Headless and headed execution

Example

```
import { test, expect } from '@playwright/test';
```

```
test('login test', async ({ page }) => {  
  await page.goto('https://example.com');
```

```
  await page.fill('#username', 'admin');  
  await page.fill('#password', 'admin123');
```

```
  await page.click('#login');
```

```
  await expect(page).toHaveURL(/dashboard/);  
});
```

2. Why is Playwright preferred over Selenium?

Feature	Playwright	Selenium
Speed	Faster	Slower
Auto Wait	Built-in	Need explicit waits
Browser Support	Chromium, Firefox, WebKit	Multiple browsers
Parallel Execution	Easy	Requires Grid/TestNG
Network Mocking	Built-in	Limited
Multiple Tabs	Easy handling	Complex
Setup	Simple	More configuration

Interview Answer

“Playwright is preferred because it provides faster execution, built-in auto waiting, better handling of modern web applications, easy parallel execution, and supports multiple browsers with a single API.”

3. What is auto-wait in Playwright?

Playwright automatically waits for elements before performing actions.

It waits for:

- Element to be visible
- Element to be attached
- Element to be stable
- Element to be enabled

Example

```
await page.click('#login');
```

Before clicking, Playwright automatically checks whether the button is ready.

Advantage

Reduces flaky tests.

4. Difference between Locator and Selector

Selector

A string used to identify elements.

```
await page.click('#login');
```

Locator

An object that finds elements dynamically.

```
const loginBtn = page.locator('#login');  
await loginBtn.click();
```

Why Locator is Better?

- Reusable
- Auto waiting
- Better stability
- Recommended approach

Interview Answer

“Locator is preferred because it supports auto waiting and re-querying the DOM whenever action is performed.”

5. What are the different locator strategies in Playwright?

By ID

```
page.locator('#username')
```

By Text

```
page.getByText('Login')
```

By Role

```
page.getByRole('button', { name: 'Login' })
```

By Label

```
page.getByLabel('Email')
```

By Placeholder

```
page.getByPlaceholder('Enter username')
```

By Test ID

```
page.getByTestId('submit-btn')
```

Best Practice

Prefer:

1. Role
2. Label
3. TestId

Avoid complex XPath.

6. What is the difference between fill() and type()?

fill()	type()
Clears existing text	Does not clear
Faster	Slower
Sets value directly	Types character by character

Example

```
await page.fill('#user', 'admin');  
await page.type('#user', 'admin');
```

When to use type()?

When testing:

- Keyboard events
 - Auto-suggestions
 - Typing delay behavior
-

7. Explain Playwright Test Runner

Playwright Test is the built-in framework for executing tests.

Features

- Parallel execution
- Retry mechanism
- Fixtures
- HTML reports
- Screenshots/videos
- Hooks
- Tagging

Example

```
test('sample test', async ({ page }) => {  
  await page.goto('https://example.com');  
});
```

8. What are Fixtures in Playwright?

Fixtures are reusable setup objects provided to tests.

Built-in Fixtures

- page
- browser
- context

Example

```
test('example', async ({ page }) => {  
  await page.goto('https://google.com');  
});
```

page is a fixture.

9. What is Browser Context?

Browser Context is an isolated browser session.

Each context has:

- Separate cookies
- Separate local storage
- Separate session

Example

```
const context = await browser.newContext();  
const page = await context.newPage();
```

Use Case

Multi-user testing.

10. Difference between Browser and Context

Browser	Context
Actual browser instance	Separate session inside browser
Heavyweight	Lightweight
Launch once	Create multiple

Example

```
const browser = await chromium.launch();
```

```
const context1 = await browser.newContext();
```

```
const context2 = await browser.newContext();
```

11. What are Hooks in Playwright?

Hooks are used for setup and teardown.

Types

- beforeAll
- beforeEach
- afterEach
- afterAll

Example

```
test.beforeEach(async ({ page }) => {  
  await page.goto('https://example.com');  
});
```

12. Difference between beforeAll and beforeEach

beforeAll	beforeEach
------------------	-------------------

Runs once	Runs before every test
-----------	------------------------

Faster	Repeated execution
--------	--------------------

Use Cases

beforeAll:

- Login once
- DB setup

beforeEach:

- Fresh setup for every test
-

13. How to handle dropdown in Playwright?

Static Dropdown

```
await page.selectOption('#country', 'India');
```

Multiple Selection

```
await page.selectOption('#country', ['India', 'USA']);
```

14. How to handle alerts?

Simple Alert

```
page.on('dialog', async dialog => {  
  console.log(dialog.message());  
  await dialog.accept();  
});
```

Dismiss Alert

```
await dialog.dismiss();
```

15. How to upload files in Playwright?

```
await page.setInputFiles('#upload', 'test.pdf');
```

Multiple Files

```
await page.setInputFiles('#upload', [  
  'a.pdf',  
  'b.pdf'  
]);
```

16. How to handle multiple tabs/windows?

```
const [newPage] = await Promise.all([  
  context.waitForEvent('page'),  
  page.click('#open')  
]);
```

```
await newPage.waitForLoadState();
```

17. What is Page Object Model (POM)?

POM is a design pattern where:

- Locators
- Methods
- Actions

are stored in separate classes.

Advantages

- Reusability
- Maintainability
- Reduced duplication

Example

```
export class LoginPage {  
  
  constructor(page) {  
    this.page = page;  
  }  
  
  async login(user, pass) {  
    await this.page.fill('#user', user);  
    await this.page.fill('#pass', pass);  
  }  
}
```

18. What is await in Playwright?

Playwright operations are asynchronous.

await waits until action completes.

Example

```
await page.click('#login');
```

Without await, execution may continue before action completes.

19. Difference between synchronous and asynchronous programming

Synchronous	Asynchronous
--------------------	---------------------

Executes one by one	Non-blocking
---------------------	--------------

Waits for completion	Continues execution
----------------------	---------------------

JavaScript and Playwright mainly use asynchronous operations.

20. What is Promise in JavaScript?

Promise represents future completion of an operation.

States

- Pending
- Resolved
- Rejected

Example

```
const data = await page.title();
```

21. How to take screenshot in Playwright?

Full Page

```
await page.screenshot({  
  path: 'page.png',  
  fullPage: true  
});
```

Element Screenshot

```
await page.locator('#logo').screenshot({  
  path: 'logo.png'  
});
```

22. How to capture video?

In config:

```
use: {  
  video: 'on'  
}
```

23. How to generate reports?

`npx playwright show-report`

HTML Report

Generated automatically after execution.

24. How to run tests in headed mode?

`npx playwright test --headed`

25. How to run a single test?

By file

`npx playwright test login.spec.ts`

By title

`npx playwright test -g "login test"`

26. How to run tests in parallel?

In config:

`workers: 4`

Or:

`npx playwright test --workers=4`

27. What is flaky test?

A flaky test:

- Sometimes passes
- Sometimes fails

without code changes.

Reasons

- Hard waits
- Timing issues
- Dynamic elements
- Network delays

Prevention

- Use locators
 - Avoid `waitForTimeout()`
 - Use assertions properly
-

28. Why should we avoid `waitForTimeout()`?

```
await page.waitForTimeout(5000);
```

Problems

- Slow execution
- Flaky tests
- Unnecessary waiting

Better Alternative

```
await expect(locator).toBeVisible();
```

29. Difference between `locator()` and `$$`

<code>locator()</code>	<code>\$\$</code>
Auto waits	No auto wait
Recommended	Old approach
Dynamic	Static array

30. Explain assertions in Playwright

Assertions validate expected results.

Example

```
await expect(page).toHaveTitle('Dashboard');
```

Common Assertions

```
await expect(locator).toBeVisible();
```

```
await expect(locator).toBeEnabled();
```

```
await expect(locator).toContainText('Success');
```

```
await expect(page).toHaveURL(/dashboard/);
```

31. How to handle authentication in Playwright?

Storage State Method

Save Login

```
await context.storageState({  
  path: 'auth.json'  
});
```

Reuse Login

```
use: {  
  storageState: 'auth.json'  
}
```

Advantage

Avoids repeated login.

32. What is network interception?

Intercepting API/network requests.

Example

```
await page.route('**/api/users', route => {  
  route.fulfill({  
    status: 200,  
    body: JSON.stringify({ name: 'Priya' })  
  });  
});
```

33. What is API Testing in Playwright?

Playwright supports API testing using request fixture.

Example

```
test('API test', async ({ request }) => {  
  
  const response = await request.get('/users');  
  
  expect(response.status()).toBe(200);
```

```
});
```

34. Explain retries in Playwright

Retries rerun failed tests automatically.

Config

retries: 2

Useful for unstable environments.

35. What are annotations/tags?

Used to categorize tests.

Example

```
test('@smoke login test', async ({ page }) => {
```

```
});
```

Run:

```
npx playwright test --grep @smoke
```

36. Explain Playwright framework structure

Typical structure:

project

├─ tests

├─ pages

├─ utils

├─ fixtures

├─ playwright.config.ts

└─ package.json

Explanation

- tests → test files
- pages → page objects
- utils → reusable methods
- fixtures → custom fixtures

- config → framework configuration
-

37. How do you avoid flaky tests?

Best Practices

- Use proper locators
- Avoid hard waits
- Use auto-wait
- Use assertions
- Mock unstable APIs
- Use retries carefully
- Isolate tests

Interview Answer

“To avoid flaky tests, I use stable locators, explicit assertions, avoid hard waits, ensure test independence, and leverage Playwright’s auto-waiting capability.”

38. What is trace viewer?

Trace Viewer helps debug failed tests.

Enable

```
use: {  
  trace: 'on'  
}
```

Open

```
npx playwright show-trace trace.zip
```

It shows:

- Actions
 - Network
 - Console
 - Screenshots
-

39. What is the difference between nth(), first(), and last()?

nth()

locator.nth(2)

first()

locator.first()

last()

locator.last()

Used when multiple elements match.

40. Explain soft assertion

Soft assertion continues execution even if assertion fails.

Example

```
await expect.soft(locator).toBeVisible();
```

Useful when validating multiple fields.

41. How to handle frames in Playwright?

```
const frame = page.frameLocator('#frame');
```

```
await frame.locator('#email').fill('test');
```

42. Difference between page and locator

page

locator

Represents webpage Represents element

Used for navigation Used for element actions

43. What is headless mode?

Browser runs without UI.

Example

```
headless: true
```

Advantages

- Faster
- Suitable for CI/CD

44. Explain CI/CD integration

GitHub Actions, Jenkins, and Azure DevOps can execute Playwright tests automatically during deployment pipelines.

45. Explain custom fixtures

Custom fixtures are reusable setup logic.

Example

```
import { test as base } from '@playwright/test';
```

```
export const test = base.extend({  
  loginPage: async ({ page }, use) => {  
    await use(new LoginPage(page));  
  }  
});
```

46. What is the difference between toHaveText() and toContainText()?

toHaveText() toContainText()

Exact text Partial text

Example

```
await expect(locator).toHaveText('Welcome');  
await expect(locator).toContainText('Wel');
```

47. Explain Promise.all() in Playwright

Runs multiple async operations together.

Example

```
await Promise.all([  
  page.waitForNavigation(),  
  page.click('#submit')  
]);
```

Used to avoid race conditions.

48. How do you debug Playwright tests?

Methods

- `page.pause()`
- Trace Viewer
- UI Mode
- Console logs
- Screenshots
- Videos

Example

```
await page.pause();
```

49. What is UI Mode?

Interactive mode for debugging tests.

```
npx playwright test --ui
```

50. Explain real-time challenges faced in Playwright

Common Challenges

- Dynamic elements
- iFrames
- Shadow DOM
- File uploads
- OTP login
- Captcha
- Flaky APIs

Solutions

- Stable locators
- Frame locators
- API mocking
- Storage state reuse
- Retry logic

